Installation Guide — Khazane-DZ

> *Intelligent Stock Management Application — Sector-Agnostic*

Table of Contents

1. Prerequisites
2. Installation – Development Mode
3. Installation – Production Mode (Docker)
4. Installation – Standalone Desktop Application
5. Verifying the Installation
6. Troubleshooting

---

1. Prerequisites

Required Software

Software	Minimum Version	Download
Node.js	18.x (LTS)	https://nodejs.org/
npm	9.x (bundled)	Included with Node.js
Docker	24.x	https://www.docker.com/products/docker-desktop
Git	2.x	https://git-scm.com/

Verifying Prerequisites

Open a PowerShell terminal and run:

```
node --version      # should display v18.x or higher
npm --version       # should display 9.x or higher
docker --version    # should display 24.x or higher
git --version       # should display 2.x or higher
```

Ports Used

Service	Port	Note
PostgreSQL	5433	Docker container (mapped from 5432)
Redis	6379	Docker container
Backend API	3002	NestJS
Frontend	5173	Vite dev server (development mode)

> ⚠️ *Make sure these ports are not already in use on your machine.*

---

2. Installation – Development Mode

Step 1: Clone the Project

```
git clone <repository-url> Khazane-DZ
cd Khazane-DZ
```

Step 2: Start Docker Containers (PostgreSQL + Redis)

```
npm run docker:up
```

This starts:

- **PostgreSQL 16** on port `5433` (user: `khazane`, password: `khazane_secret`, database: `khazane_db`)
- **Redis 7** on port `6379`

Verify that containers are running:

```
docker ps
```

You should see `khazane-postgres` and `khazane-redis` with status `Up`.

Step 3: Install Backend Dependencies

```
cd backend
npm install
```

Step 4: Configure Environment Variables

The `backend/.env` file is pre-configured:

```
# Database
DATABASE_URL=postgresql://khazane:khazane_secret@localhost:5433/khazane_db

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# JWT
JWT_SECRET=change-me-in-production
JWT_EXPIRES_IN=1d

# Application
PORT=3002
NODE_ENV=development
```

> ⚠️ ***In production**, you must change `JWT_SECRET` to a secure random value and update the database passwords.*

Step 5: Run Database Migrations

```
npx prisma migrate dev
```

This command creates all tables in PostgreSQL and generates the Prisma client.

Step 6: Seed the Database

```
npx prisma db seed
```

Data created by the seed:

- **Admin user**: `admin@khazane.dz` / password `admin123`
- **Site**: "Entrepôt Principal" (code `ENT-01`), type Warehouse, address "Zone industrielle, Alger"
- **Zone**: "Allée 1" (code `A1`), type AISLE
- **5 locations**: `A1-01` to `A1-05` (Shelf 1 to 5), max capacity 100
- **Category**: "Général"
- **Assignment**: Admin assigned to main site

Step 7: Start the Backend

```
npm run start:dev
```

The API server starts at `http://localhost:3002`.
Swagger documentation is available at `http://localhost:3002/api`.

Step 8: Install Frontend Dependencies

Open a **new terminal**:

```
cd frontend
npm install
```

Step 9: Start the Frontend

```
npm run dev
```

The frontend starts at `http://localhost:5173`.

Summary – Quick Commands (from root)

```
npm run docker:up          # 1. Start PostgreSQL + Redis
cd backend && npm install  # 2. Backend dependencies
npx prisma migrate dev     # 3. Migrations
npx prisma db seed         # 4. Seed data
npm run start:dev          # 5. Start backend
# In a new terminal:
```

```
cd frontend && npm install # 6. Frontend dependencies
npm run dev                # 7. Start frontend
```

---

3. Installation – Production Mode (Docker)

Build the Frontend

```
cd frontend
npm ci
npm run build
```

The build is generated in `frontend/dist/`.

Build the Backend

```
cd backend
npm ci
npx nest build
```

The build is generated in `backend/dist/`.

Start the Backend in Production Mode

```
cd backend
$env:NODE_ENV="production"
$env:DATABASE_URL="postgresql://khazane:khazane_secret@localhost:5433/khazane_db"
$env:JWT_SECRET="your-secure-production-secret"
$env:PORT="3002"
$env:SERVE_STATIC="true"
node dist/main.js
```

With `SERVE_STATIC=true`, the backend automatically serves the compiled frontend. The complete application is then accessible at `http://localhost:3002`.

---

4. Installation – Standalone Desktop Application

The desktop application is a standalone Windows executable that embeds PostgreSQL, the backend, and the frontend.

Additional Prerequisites

- Internet connection (for initial download of portable PostgreSQL ~200 MB)
- Approximately 1 GB of free disk space

Launch the Automated Build

From the project root:

```
.\scripts\build-desktop.ps1
```

This script performs 6 steps automatically:

Step	Action
1/6	Download portable PostgreSQL 16
2/6	Build the React frontend
3/6	Build the NestJS backend
4/6	Compile the seed script
5/6	Install Electron dependencies
6/6	Package with electron-builder (NSIS)

Script Options

```
# Skip PostgreSQL download (if already present)
.\scripts\build-desktop.ps1 -SkipPgDownload

# Generate an unpacked folder (for testing)
.\scripts\build-desktop.ps1 -DirOnly
```

Result

The Windows installer (`.exe`) is generated in the `dist-desktop/` folder.

Installation on the Client Machine

1. Run the `.exe` installer
2. Follow the installation wizard
3. The application starts with a splash screen while initializing the embedded database
4. No PostgreSQL or Node.js installation is required on the client machine

---

5. Verifying the Installation

Login Test

1. Open the browser at `http://localhost:5173` (dev) or `http://localhost:3002` (production)
2. Log in with:
   - **Email**: `admin@khazane.dz`
   - **Password**: `admin123`
3. You should access the Dashboard

API Test

```
# Get a JWT token
$body = '{"email":"admin@khazane.dz","password":"admin123"}'
```

```
$response = Invoke-RestMethod -Uri http://localhost:3002/api/v1/auth/login -Method POST -
Body $body -ContentType "application/json"
$token = $response.access_token

# Test the products API
Invoke-RestMethod -Uri http://localhost:3002/api/v1/products -Headers @{ Authorization =
"Bearer $token" }
```

Useful Tools

```
# Open Prisma Studio (visual database interface)
npm run db:studio

# View Docker container logs
docker logs khazane-postgres
docker logs khazane-redis
```

---

6. Troubleshooting

Port 5433 is Already in Use

```
# Find the process using the port
netstat -ano | findstr :5433
# Stop the existing container
docker stop khazane-postgres
docker rm khazane-postgres
# Restart
npm run docker:up
```

Prisma Error "Can't reach database server"

Make sure Docker is running and the PostgreSQL container is up:

```
docker ps | findstr khazane-postgres
```

If the container is not listed:

```
npm run docker:up
```

Backend Won't Start (Port 3002 Occupied)

```
netstat -ano | findstr :3002
taskkill /PID <pid_number> /F
```

Reset the Database

```
cd backend
npx prisma migrate reset
npx prisma db seed
```

> ⚠️ *This operation deletes all existing data.*

Regenerate the Prisma Client

If you get Prisma-related errors after a schema change:

```
cd backend
npx prisma generate
```

Document generated for Khazane-DZ v0.1.0